Maximum Likelihood Reinforcement Learning

> #RL

- What problem does this framework solve?
  - Given a very hard problem P_h. Let a model generate N responses, and the possibility of correct answer is p.
  - For Normal RL, we reward each response ($\{0, 1\}$), calculate the advantage for each of the answers, and then *find the average of advantages over the* Nresponses as the train signal.
  - When $p$is small (# of correct answers is small), because of the division over N, the train signal (gradient) is even smaller. *The hard the problem is, the slower the model will evolve.*
  - In traditional RL, maximizing the expected reward can be expressed as maximizing the probability of gaining a right answer

    $J_{RL}(\theta) = p$

    The gradient is:

    $\nabla_\theta J_{RL} = \nabla_\theta p$

    If $p$is small, then its gradient $\nabla_\theta p$is also small (as mentioned above, it diminishes during finding the mean).
- What does this framework do?
  - Qualitatively speaking, this framework wants to maximize the likelihood. That is to say, even if the probability of a good answer is small, *it happens*. We want to find a distribution (policy model) that **ALIGNs with our findings**.
  - Formally, the objective function becomes:

    $J_{ML}(\theta) = \log(p)$

    The gradient is:

    $\nabla_\theta J_{ML} = \nabla_\theta \log(p) = \frac{1}{p} \nabla_\theta p$

    When $p$is small but not 0, because of $\frac{1}{p}$, the gradient will be amplified significantly to achieve our goal: maximize what we have seen. *The hard the problem is, the faster the model will evolve.*
  - [Mathematics] The Maclaurin Expansion
    - Given that

      $J_{ML}(x) = \log p = \log(1 - (1 - p)) = \ln(1 - (1 - p))$,

      and we know that the Maclaurin:

      $\ln(1 - u) = -\sum_{k=1}^{\infty} \frac{u^k}{k}$

      Let $1 - p$ be the probability of failure, then $p = 1 - (1 - p)$. Bring this to the Maclaurin, we have

      $\ln(1 - u) = -\sum_{k=1}^{\infty} \frac{u^k}{k}, \text{ where } u = (1 - p)$,

      we have:

      $J_{ML}(x) = -\sum_{k=1}^{\infty} \frac{(1-p)^k}{k}$
    - Here:

        - $(1-p)^k$= all $k$times are failed, we call this $\text{fail@}k(x) = (1-p)^k$.
        - Then, on the opposite side, $\text{pass@}k(x) = 1-(1-p)^k$.
    - Differentiate $J_{ML}(x) = -\sum_{k=1}^{\infty} \frac{(1-p)^k}{k}$we have:

      $\nabla_\theta J_{ML}(x) = \sum_{k=1}^{\infty} \frac{1}{k} \nabla_\theta(\text{pass@}k(x))$ *(This will be used in the proof of theorem 2; details in Appendix B of the original paper.)*
    - The ML objective is actually the weighted sum of countless Pass@k objectives. As k increases, the weight $\frac{1}{k}$ decreases, yet it encapsulates higher-order success information.
    - The traditional RL objective is simply the first term of this series (when $k=1$, $\nabla\text{pass@}1 = \nabla p$).
- MaxRL Algorithm
    - We don't want the ∞ term in the series, so we replace it with a finite number T.

      $J_{MaxRL}^{(T)}(x) = -\sum_{k=1}^{T} \frac{(1-p)^k}{k}$
    - However, we don't use the $-\sum_{k=1}^{T} \frac{(1-p)^k}{k}$ to calculate the gradient.
    - Instead, we use: *(proof)*

      $\nabla_\theta J_{ML}(x) = \mathbb{E}[\nabla_\theta \log m_\theta(z|x) \mid f(z) = y^*(x)]$

      Meaning: we only calculate the average of the gradient for those correct answers. Here:
        - $z$is the latent variable of inference such as CoT
        - $f$is a decoder for z, such as the policy model. $f(z)$ is the final result given by the policy model.
        - $y^*(x)$ is the ground truth of question x.
    - Based on this gradient, we can write out a gradient estimator. Let Kbe # of success, and $S_i = \nabla_\theta \log m_\theta(z_i|x)$ be the gradient of the i-th sample, then we have:

      $\hat{g}_{MaxRL} \approx \frac{1}{K} \sum_{i \in \text{Success}} S_i$

      Expand the $\sum_{i \in \text{Success}}$, we have $\tilde{g}_N(x) = \frac{1}{K} \sum_{i=1}^{N} r_i S_i$, as $r_i = 0$ if not succeeded.

      To decrease the variance, we minus the expected gradient of ALL samples:

      $\tilde{g}_N(x) = \frac{1}{K} \sum_{i=1}^{N} r_i S_i - \frac{1}{N} \sum_{i=1}^{N} S_i$

Algorithm 1 On-Policy Implementation of MaxRL.

require Batch of inputs B, number of rollouts N, latent policy $m_\theta(\cdot \mid \cdot)$

for each input $x \in B$ **do**
  Sample $z_1, \ldots, z_N \sim m_\theta(\cdot \mid x)$
  for $j = 1$ to N **do**
    $r_j \leftarrow \mathbb{I}\{f(z_j) = y^*(x)\}$
    $S_j \leftarrow \nabla_\theta \log m_\theta(z_j \mid x)$
  end for
  $\widehat{r}(x) \leftarrow \frac{1}{N} \sum_{j=1}^{N} r_j$
  $$\widehat{g}(x) \leftarrow \begin{cases} \dfrac{1}{N\,\widehat{r}(x)} \displaystyle\sum_{j=1}^{N} (r_j - \widehat{r}(x)) S_j, & \widehat{r}(x) > 0, \\ 0, & \text{otherwise} \end{cases}$$
end for
$\widehat{g} \leftarrow \frac{1}{|B|} \sum_{x \in B} \widehat{g}(x)$
return $\widehat{g}$

- Where is $\frac{1}{p}$? *(proof)*

- My Thoughts
  - One viable application is for training tools. However, training directly in real-world interactive environments is prohibitively expensive (due to the sampling requirements).
  - Thus, we can use a textual environment for simulation. Enable the agent to interact within a text-based environment, where the environment returns a binary outcome (success or failure) for each task and trajectory. Then use this algorithm for optimization. Since tool usage is a highly challenging task, it is well-suited for this maximum likelihood estimation-based RL approach.
  - To mitigate illusions, we should incorporate a small amount of real-world interaction (similar to the second usage mentioned here).
- Work Cited
  - Paper
  - Chat History